1. Linear Search — First Element With Maximum Distance

Given an unsorted array, find the element whose **first and last occurrence have the maximum distance**.

Distance:

```
None
lastIndex - firstIndex
```

If multiple elements have the same maximum distance, return the element whose **first occurrence comes earliest**.

Input

```
None
arr = [5, 2, 8, 5, 2, 9, 8, 5]
```

Output

```
None
5
```

Restrictions

- Must use Linear Search.
- Do not sort.
- No Dictionary/HashMap.
- No extra array.
- Handle duplicate values.
- Explain your time complexity.

---

2. Linear Search — Longest Consecutive Increasing Segment

Find the length of the longest contiguous segment where every next element is greater than the previous element.

Input

```
None
arr = [10, 12, 15, 3, 5, 7, 8, 2, 4]
```

Output

```
None
4
```

Because:

```
None
3, 5, 7, 8
```

Restrictions

- One traversal only.
- `O(N)` time.
- `O(1)` extra space.
- Do not sort.

---

3. Linear Search — Majority Element Verification

You are given an unsorted array.

Determine whether a majority element exists.

A majority element occurs **more than N/2 times**.

Input

```
None
arr = [2, 2, 1, 1, 1, 2, 2]
```

Output

```
None
2
```

Hard Restrictions

- First determine a candidate using constant space.
- Then verify the candidate.
- Do not sort.
- Do not use HashMap.
- `O(N)` time.
- `O(1)` extra space.

---

4. Binary Search — First Element ≥ X

Given a sorted array containing duplicates, find the index of the **first element greater than or equal to X**.

Input

```
None
arr = [2, 4, 4, 4, 7, 9, 12]
X = 4
```

Output

```
None
1
```

Restrictions

- Binary Search only.

- `O(log N)`.
- No Linear Search.
- Must work when X is smaller than every element.
- Must work when X is larger than every element.

---

5. Binary Search — Last Element ≤ X

Find the index of the **last element smaller than or equal to X**.

Input

```
None
arr = [2, 4, 4, 4, 7, 9, 12]
X = 8
```

Output

```
None
4
```

---

6. Binary Search — Minimum Capacity Problem

A company needs to ship packages in `D` days.

Packages must be shipped **in the given order**.

Find the minimum truck capacity required.

Input

```
None
weights = [10, 20, 30, 40, 50]
D = 2
```

Output

```
None
90
```

One possible division:

```
None
10 + 20 + 30 = 60
40 + 50 = 90
```

Restrictions

- Must use Binary Search on the answer.
- Do not rearrange packages.
- Do not use brute force.
- Explain the search space.

---

7. Binary Search — Allocate Books

There are `N` books arranged in order.

Each book has a number of pages.

Allocate contiguous books to `K` students such that the **maximum pages assigned to any student is minimized**.

Input

```
None
pages = [10, 20, 30, 40]
K = 2
```

Output

```
None
60
```

Possible allocation:

```
None
Student 1 → 10 + 20 + 30 = 60
Student 2 → 40
```

Restrictions

- Binary Search on answer.
- Contiguous allocation only.
- Every student must receive at least one book.
- Do not sort.

---

8. Binary Search — Peak in Mountain Array

A mountain array strictly increases and then strictly decreases.

Find the peak index.

Input

```
None
arr = [1, 3, 7, 12, 9, 5, 2]
```

Output

```
None
3
```

Restrictions

- `O(log N)`.
- Binary Search.
- No Linear Search.

---

9. Binary Search — Rotated Array With Duplicates

Search for a target in a rotated sorted array that **may contain duplicates**.

Input

```
None
arr = [2, 5, 6, 0, 0, 1, 2]
target = 0
```

Output

```
None
3
```

Restrictions

- Use Binary Search.
- Return any valid index.
- Do not sort.
- Do not use Linear Search intentionally.
- Explain what happens when:

```
None
arr[left] == arr[mid] == arr[right]
```

---

10. Binary Search — Kth Missing Positive Number

Given a sorted array of positive integers, find the Kth positive integer that is missing.

Input

```
None
arr = [2, 3, 4, 7, 11]
K = 5
```

Output

```
None
9
```

Missing values:

```
None
1, 5, 6, 8, 9
```

Restrictions

- Binary Search.
- `O(log N)`.
- No extra array.
- Do not generate all missing values.

---

11. Bubble Sort — Inversion Count

Using Bubble Sort logic, determine how many swaps are required to sort the array.

Input

```
None
arr = [8, 4, 2, 1]
```

Output

```
None
6
```

Restrictions

- Bubble Sort only.
- Count every swap.
- Do not use Merge Sort.
- Do not use another array.

Interview Question

Why is the number of Bubble Sort swaps equal to the number of inversions?

---

12. Bubble Sort — Adaptive Performance

Given:

```
None
arr = [1, 2, 3, 5, 4, 6, 7, 8]
```

Perform optimized Bubble Sort.

Determine:

```
None
Number of passes
Number of comparisons
Number of swaps
```

Then repeat for:

```
None
[8, 7, 6, 5, 4, 3, 2, 1]
```

Explain the difference.

---

13. Selection Sort — Kth Smallest Without Full Sorting

Find the **5th smallest distinct element**.

Input

```
None
arr = [20, 5, 10, 5, 30, 15, 20, 40, 25]
K = 5
```

Restrictions

- Selection Sort logic only.
- Do not completely sort.
- No HashSet.
- No extra array.
- Duplicates count once.

14.Selection Sort — Minimum Number of Swaps

Sort the array using Selection Sort and calculate the exact number of swaps.

```
None
arr = [4, 3, 2, 1, 5]
```

Determine:

```
None
Sorted array = ?
Comparisons = ?
Swaps = ?
```

Then determine whether the same array would require more or fewer swaps with Bubble Sort.

15.Selection Sort — Partial Sorting

You only need the **three smallest elements in sorted order**.

```
None
arr = [29, 10, 14, 37, 13, 5, 18, 2]
K = 3
```

Expected result

```
None
2 5 10
```

Restrictions

- Selection Sort logic.
- Stop after K selections.
- Do not completely sort the array.
- `O(N × K)` maximum comparisons.

---

16. Insertion Sort — Count Inversions

Use the behavior of Insertion Sort to determine the number of inversions.

Input

```
None
arr = [5, 3, 4, 2, 1]
```

Output

```
None
?
```

An inversion exists when:

```
None
i < j && arr[i] > arr[j]
```

Requirements

- Use Insertion Sort logic.
- Count shifts.
- Explain the relationship between shifts and inversions.

17.Insertion Sort — Stable Multi-Criteria Sorting

Employee records:

```
None
ID   Department   Salary
101  IT           50000
102  HR           50000
103  IT           30000
104  HR           30000
105  IT           50000
```

Sort according to:

1. Salary ascending.
2. If salary is equal, Department alphabetically.
3. If both are equal, preserve original order.

Requirements

- Insertion Sort only.
- Stable.
- No built-in sorting.
- No additional array.